INDIVIDUAL NLP CAPSTONE

HausaLens

Lightweight sentiment classification for Hausa social-media text

A reproducible comparison of six supervised classifiers using the human-annotated NaijaSenti corpus, with held-out evaluation, error analysis, and an interactive deployment.

Project type · Text Classification for an African Language
Language · Hausa
Task · Positive / Neutral / Negative sentiment
Report date · 17 October 2026

Project links

GitHub repository: https://github.com/Mahamatbt/summative

Live system: https://mahamatbt.github.io/summative/

Demo video: https://youtu.be/g8qN6C-GF6k

The public system is hosted on GitHub Pages. The repository includes the trained Python model and local Streamlit instructions. The student's recorded demonstration is available through the YouTube link above.

Abstract

This project investigates three-class sentiment classification for informal Hausa social-media text. The study uses 22,152 naturally occurring, human-annotated tweets from the open NaijaSenti release: 14,172 training, 2,677 validation, and 5,303 test items. Six configurations compare a majority baseline and word TF-IDF with Multinomial Naive Bayes against word-, character-, and hybrid TF-IDF representations with linear support-vector machines. Character 3-5 grams achieved the best original validation macro-F1 (0.7698), followed by 0.7658 test accuracy and 0.7649 test macro-F1 after refitting on train plus validation. A quality audit found 215 train/validation text overlaps; an overlap-free validation sensitivity check still ranks the character model first at 0.7544. No normalized training/test text overlap was found. Positive sentiment was easiest (F1 0.8446), while neutral was hardest (F1 0.7044). The model is publicly deployed through a browser interface using exported learned parameters, with numerical parity verified against Python. A local Streamlit alternative is retained. The findings support a compact Hausa sentiment baseline while exposing clear limitations in contextual interpretation.

22,152	0.7649	1.3 MB	0
HUMAN-ANNOTATED ITEMS	TEST MACRO-F1	MODEL ARTIFACT	EXTERNAL API CALLS

Contents

- 1. Introduction
- 2. Related Work
- 3. Dataset and Data Preparation
- 4. Methodology
- 5. Experiments and Results
- 6. Error Analysis and Discussion
- 7. Deployment
- 8. Limitations and Future Work
- 9. Conclusion
- 10. References
- Appendix A. Reproducibility and Individual Contribution

1. Introduction

Sentiment classification assigns an opinion-bearing text to categories such as positive, neutral, or negative. It is useful for summarizing public feedback, monitoring service quality, and studying online discussion, but most datasets and production tools prioritize high-resource languages. NaijaSenti was introduced to address this imbalance for Hausa, Igbo, Nigerian Pidgin, and Yorùbá, using manually annotated Twitter data and documenting extensive multilingual and code-mixed usage [1]. AfriSenti later broadened shared-task evaluation to 14 African languages [2].

Hausa is a consequential case because users produce substantial informal text online, yet spelling variation, abbreviated writing, borrowed English expressions, religious phrases, emojis, and missing conversational context complicate sentiment analysis. The problem is not merely whether a classifier can return a label; it is whether the complete workflow is reproducible, whether model selection is isolated from test evaluation, and whether failures are examined rather than hidden behind a single score.

The intended users are researchers, students, and teams exploring aggregate sentiment in Hausa feedback. The application supports low-stakes research and learning. Generalization to customer service, news, or other domains is a motivation for future evaluation, rather than an established result. This is the text-classification option; the assignment's restriction on selecting an unsupported translation direction applies to its separate machine-translation option.

Objective and research questions

- RQ1 — How much does a learned classifier improve over a majority-class baseline on the fixed NaijaSenti Hausa validation split?
- RQ2 — Do character n-grams, which can represent spelling fragments and morphological variation, outperform word n-grams for this dataset?
- RQ3 — Does combining word and character features or removing emoji improve macro-F1?
- RQ4 — Which labels and surface-form conditions account for the largest share of held-out errors?

The project's own contribution is the end-to-end experimental pipeline, conservative preprocessing, six controlled comparisons, held-out evaluation, error-slice analysis, compact serialized model, local explanation method, web application, and reproducibility documentation. The underlying tweets, annotations, and published transformer benchmarks are existing resources and are cited separately.

2. Related Work

Muhammad et al. introduced NaijaSenti as a large, open sentiment resource for four Nigerian languages. Their collection procedure combined language-specific stop words, emojis, and translated sentiment lexicons to retrieve candidates, followed by human annotation. They reported substantial code-mixing and evaluated multilingual pretrained language models. On Hausa, AfriBERTa-large achieved 81.0 weighted F1 in their reported benchmark, outperforming mBERT and XLM-R-base in that table [1]. This establishes a useful upper reference, but it is not a directly replicated score here because the released files used in this project contain different split counts from the early LREC table.

The AfriSenti-SemEval shared task formalized monolingual, multilingual, and zero-shot sentiment evaluation across 14 African languages and attracted dozens of submissions. The organizers reported weighted F1 for the shared-task systems [2]. This project instead selects on macro-F1 and reports weighted F1 separately; those measures must not be conflated. Ogueji et al. investigated AfriBERTa, a compact multilingual model for African-language tasks [3]. These studies motivate a future Transformer comparison but are existing research, not models fine-tuned in this project.

Classical representations remain relevant when reproducibility, speed, and compact deployment matter. Keinan and Hacohen-Kerner tested word and character TF-IDF n-grams alongside classical and deep models in AfriSenti, demonstrating that n-gram baselines belong in a serious experimental comparison rather than being dismissed [4]. Linear support-vector machines construct maximum-margin decision boundaries in high-dimensional feature spaces [5], making them well matched to sparse TF-IDF text vectors. HausaLens therefore treats a compact linear model as a deliberate, testable design choice, not as a claim of state-of-the-art performance.

3. Dataset and Data Preparation

3.1 Source, license, and fit

The data are the Hausa split files from the official HausaNLP/NaijaSenti GitHub repository [6]. They are naturally occurring tweets annotated by people; no large-language-model-generated examples were used. The repository releases the resource under CC BY 4.0. Its three labels map exactly to the target application: negative, neutral, and positive sentiment in informal Hausa social-media text.

Split	Rows	Negative	Neutral	Positive	Median tokens
Train	14,172	4,573	4,912	4,687	11
Validation	2,677	894	896	887	11
Test	5,303	1,759	1,789	1,755	9
Total	22,152	7,226	7,597	7,329	10

Table 1. Official repository release used in this study. Counts were recomputed locally from the downloaded TSV files.

The early LREC paper lists a larger Hausa benchmark split (18,989 / 2,714 / 5,427) [1], whereas the current official files used here contain 14,172 / 2,677 / 5,303 and match later AfriSenti system descriptions. This report records the exact files and counts rather than silently blending versions. The download script checks row counts, and model outputs record the runtime environment.

3.2 Cleaning and leakage control

- Preserve the publisher's train, validation, and test boundaries; never use test labels for selection.
- Drop missing text or labels and exact duplicate rows within a split. No rows were removed in the downloaded release.
- Replace URLs and account handles with stable URL and USER tokens; lowercase and collapse whitespace.
- Retain Hausa characters, punctuation, hashtags, and emoji because they may carry sentiment.
- Fit TF-IDF vocabularies only on the current training partition inside each experiment pipeline.

The corpus is close to class balanced, so accuracy is reported but macro-F1 is the primary model-selection metric. Macro averaging gives each class equal weight and therefore exposes weak neutral-class performance that accuracy alone can conceal.

3.3 Quality audit and split overlap

The final reproducible audit found no missing text/label rows, no exact duplicate rows within a split, and no normalized duplicate texts within a split. However, training and validation share 215 normalized texts, about 8.0% of validation. Preserving publisher boundaries therefore did not make the original validation comparison fully independent. Training/test and validation/test have zero normalized text matches. This check covers exact normalized overlap, not semantic near-duplicates or shared conversational context. The original experiment is retained and the validation overlap is investigated separately in Section 5.

4. Methodology

4.1 Models

Configuration	Representation	Purpose
Majority baseline	No text features	Minimum non-learning reference
Word + NB	Word unigrams, TF-IDF	Simple probabilistic lexical baseline
Word + SVM	Word 1–2 grams, TF-IDF	Discriminative word-sequence model
Character + SVM	Character 3–5 grams, TF-IDF	Subword spelling and morphology
Hybrid − emoji	Word + character TF-IDF	Emoji ablation
Hybrid	Word + character TF-IDF	Test whether feature union helps

Table 2. Controlled configurations trained on the same training split.

TF-IDF scales an n-gram by its frequency in the current document and by the inverse frequency of documents containing it. Word models use unigrams or 1-2 grams. Character models use the scikit-learn char_wb analyzer, which creates 3-5 character sequences within padded word boundaries [7]. Terms appearing fewer than two documents for word features or three documents for character features are omitted. Sublinear term frequency is enabled.

The linear SVM uses C = 1.5, class-balanced weights, and at most 5,000 iterations. A one-versus-rest decision function is learned for the three labels. The interface applies softmax to the three margins only to create an intuitive relative-confidence display; this value is explicitly not described as a calibrated probability. Local feature evidence is computed by multiplying the input's non-zero TF-IDF values by the coefficient vector for the predicted class and ranking positive contributions.

For a feature count c, sublinear term frequency is 1 + log(c). Smoothed inverse document frequency is log((1 + N)/(1 + df)) + 1. The resulting vector is L2-normalized. Each class score is its learned intercept plus the dot product of the feature vector and class coefficients. Prediction selects the highest score. Input is a Hausa text string; output is one of three labels, margin-derived relative scores, and feature contributions. Character features are capped at 65,000; the fitted final vocabulary contains 43,806. Naive Bayes uses alpha = 0.7. These are fixed configurations, not an exhaustive hyperparameter search. No pretrained embedding or Transformer is used.

The so-called emoji-removal experiment removes characters with code points of 10,000 or above. This is an approximate high-code-point ablation, not a complete Unicode emoji detector; some emoji sequences may remain and non-emoji characters may also be removed. Its small performance difference cannot establish the causal value of emoji.

4.2 Evaluation protocol

Each candidate is fitted on 14,172 training items and evaluated on the fixed 2,677-item validation set. The configuration with the highest validation macro-F1 is selected. Only then is the winner refit on training plus validation (16,849 items) and evaluated once on the untouched 5,303-item test set. Reported metrics are accuracy, macro precision, macro recall, macro-F1, weighted-F1, per-class precision/recall/F1, and a confusion matrix. No hyperparameter search or model choice uses the test labels.

5. Experiments and Results

Model	Accuracy	Macro-P	Macro-R	Macro-F1	Train s
Majority baseline	0.3347	0.1116	0.3333	0.1672	0.00
Word unigram + NB	0.7150	0.7180	0.7152	0.7163	0.20
Word (1-2) + Linear SVM	0.7426	0.7437	0.7429	0.7432	0.51
Hybrid word+character SVM	0.7654	0.7672	0.7656	0.7663	2.01
Hybrid without emoji	0.7654	0.7676	0.7656	0.7664	2.05
Character (3-5) + Linear SVM	0.7688	0.7718	0.7689	0.7698	1.61

Table 3. Validation results. Timing is descriptive for one local run, not a hardware-normalized benchmark.

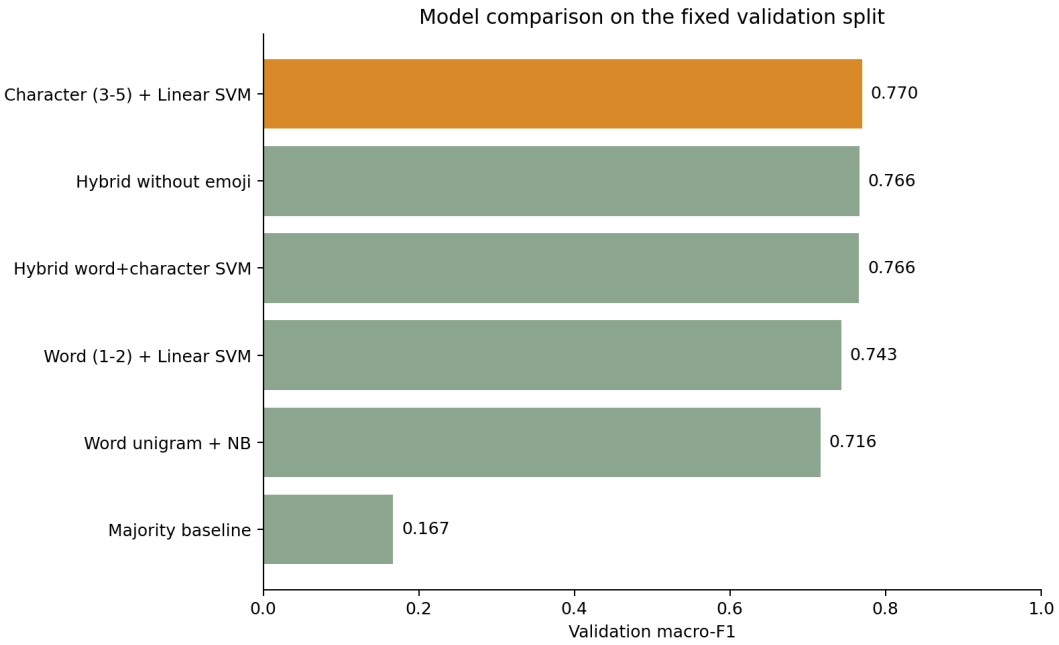


Figure 1. Validation macro-F1 for all six configurations.

All learned models substantially exceed the majority macro-F1 of 0.1672. Word-unigram Naive Bayes reaches 0.7163; replacing it with word 1-2 grams and a linear SVM adds 2.69 points. Character n-grams add another 2.66 points and win at 0.7698. The hybrid model does not improve on character features alone, and its no-emoji ablation differs by only 0.0002 macro-F1 from the full hybrid. More features were therefore not automatically better. This answers RQ1-RQ3 and justifies selecting the character model.

Validation sensitivity: remove training-text overlap

Configuration	Validation items	Macro-F1
Character (3-5) + Linear SVM	2462	0.7544
Hybrid without emoji	2462	0.7515
Hybrid word+character SVM	2462	0.7513
Word (1-2) + Linear SVM	2462	0.7262
Word unigram + NB	2462	0.7021
Majority baseline	2462	0.1729

Supplementary comparison: exclude the 215 validation texts that normalize to training texts, leaving 2,462 items, then fit the same six configurations on the original training data. Character SVM still ranks first (0.7544), ahead of the approximate no-emoji hybrid (0.7515), full hybrid (0.7513), word SVM (0.7262), Naive Bayes (0.7021), and majority baseline (0.1729). The lower scores show that the original validation result was optimistic. This is a validation-only sensitivity analysis; it does not change the already selected deployed artifact or tune against test labels. Small differences are not claimed to be statistically significant.

5.1 Held-out test performance

Class	Precision	Recall	F1	Support
Negative	0.7081	0.7874	0.7456	1,759
Neutral	0.7587	0.6574	0.7044	1,789
Positive	0.8347	0.8547	0.8446	1,755
Macro avg	0.7672	0.7665	0.7649	5,303

Table 4. Final performance on the untouched test split after refitting the selected configuration.

0.7658	0.7649	0.8446	0.7044
TEST ACCURACY	TEST MACRO-F1	POSITIVE F1	NEUTRAL F1

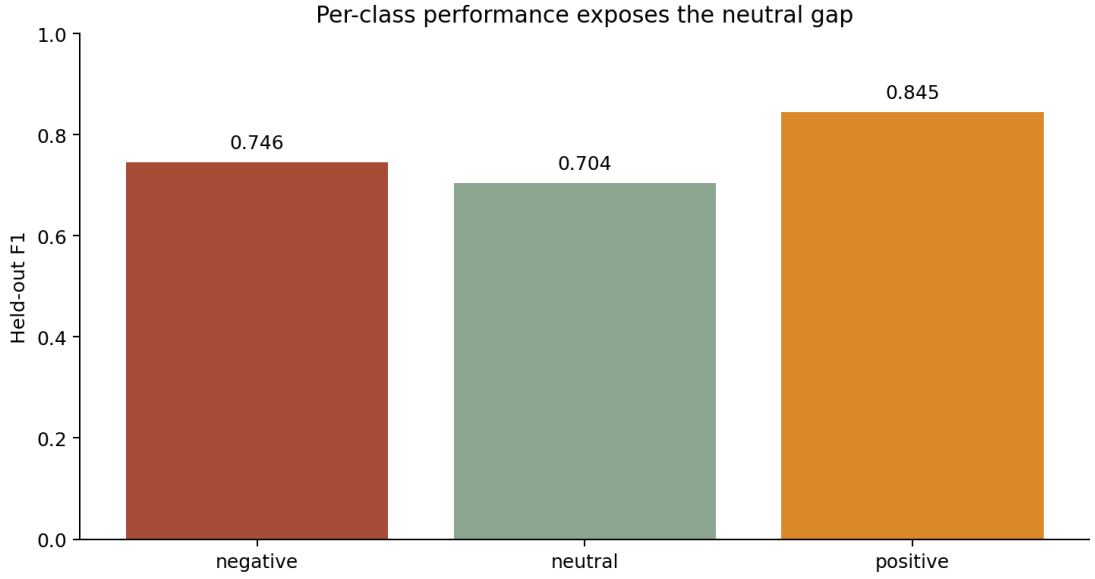


Figure 2. Held-out F1 by class. Macro-F1 makes the neutral-class weakness visible.

Positive sentiment is recognized most reliably. Neutral recall is 0.6574, meaning roughly one third of true neutral items are assigned another class. The 0.0049 gap between original validation and final test macro-F1 is not proof of stable generalization: the validation overlap and the different training sizes complicate that interpretation. Precision measures correctness among a class's predictions; recall measures coverage of its reference items. F1 = 2PR/(P + R), with zero-division conventions as in scikit-learn. Macro-F1 averages the three class F1 scores equally; weighted F1 weights them by reference support. Published Hausa Transformer figures use different data versions and sometimes different averaging, so they are context rather than a replicated head-to-head benchmark [1, 8].

6. Error Analysis and Discussion

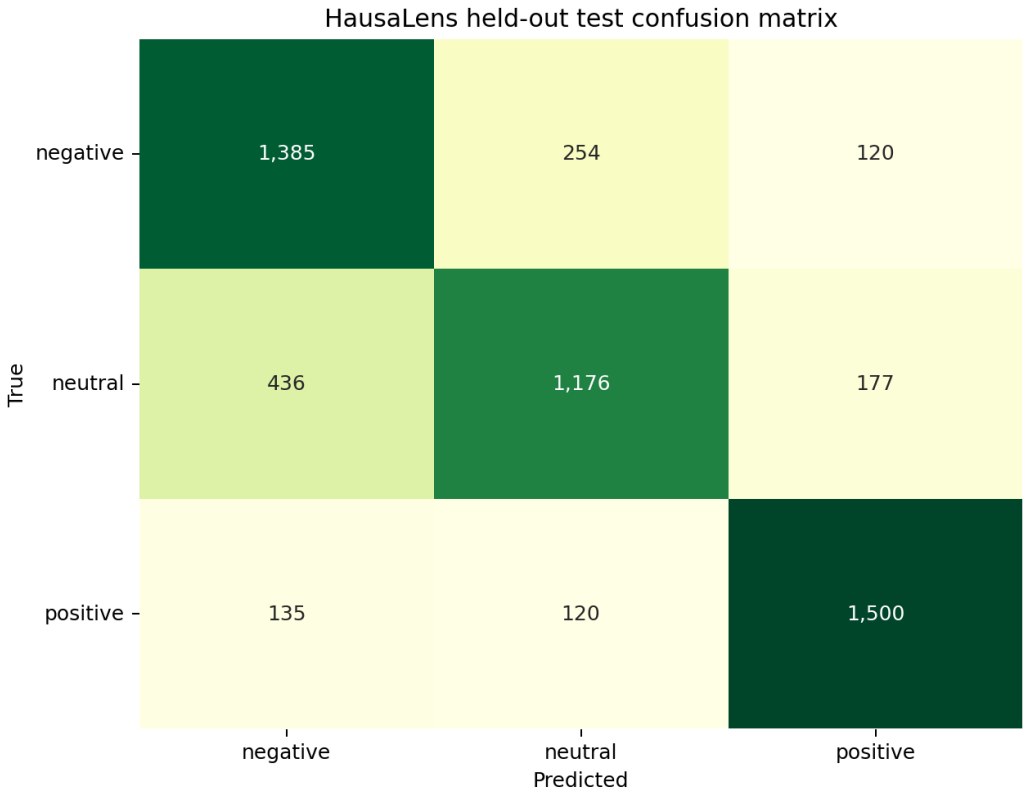


Figure 3. Held-out confusion matrix (n = 5,303).

The model makes 1,242 errors. Of these, 613 occur on neutral items. The largest single confusion is neutral → negative (436 items), followed by negative → neutral (254). This asymmetry suggests that the learned boundary more readily treats neutral statements containing negative-looking fragments or topics as negative than the reverse. Because annotation disagreement is intrinsic to sentiment, some border cases may not have a single context-free interpretation.

6.1 Descriptive slices

Surface cue	Present n	Present error	Absent error	Difference
≤5 tokens	1,235	21.3%	24.1%	-2.8%
Negation marker	970	26.9%	22.6%	+4.3%
Question cue	359	27.6%	23.1%	+4.5%
Non-ASCII character	352	19.9%	23.7%	-3.8%

Table 5. Surface-form error slices. These are descriptive indicators, not causal or validated linguistic categories.

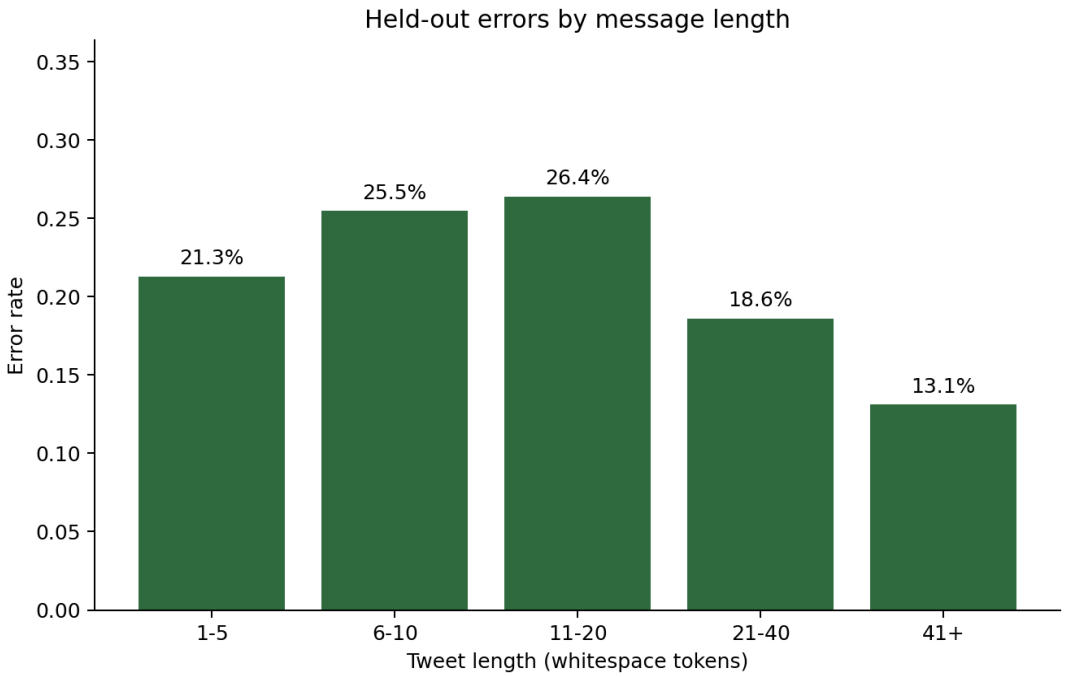


Figure 4. Error rate by whitespace-token length.

Negation-marked messages have a 26.9% error rate versus 22.6% otherwise, and question-like cues have 27.6% versus 23.1%. Contrary to a simple ‘shorter is harder’ hypothesis, messages with five or fewer whitespace tokens are slightly easier than the rest. The highest length-bin error occurs at 11-20 tokens (26.4%); messages above 40 tokens are easiest (13.1%), possibly because they provide more redundant sentiment evidence. Non-ASCII characters, a broad proxy that includes Hausa letters and emoji, are associated with a lower error rate in this sample. None of these observational differences proves causation.

6.2 Qualitative examples

Outcome	Gold → Predicted	Held-out text	Interpretation
Success	negative → negative	wannan ai karyace	Direct negative lexical cue; concise context.
Success	positive → positive	allah yah taimaka	Frequent positive prayer/wish pattern.
Success	neutral → neutral	jirgin sama mai amfani da hasken rana	Factual nominal phrase with little polarity.
Failure	negative → neutral	ni dai ban yafe ba	Negation is present but the phrase is short and context-dependent.
Failure	positive → negative	amma ai gaskiya ta faɗa	Adversative phrasing can resemble disagreement despite positive gold label.
Failure	neutral → negative	tanbayata itace meyasa yan jarida basa kamuwa da cutar corona virus	Question contains disease and negation cues; topic words may dominate neutrality.

Table 6. Examples are reproduced from the held-out test set with original spelling. English glosses are omitted because no native-speaker validation was available.

The examples point to four recurring challenges: neutral-versus-negative boundaries, explicit negation, rhetorical or conversational questions, and a lack of surrounding thread context. Character n-grams are effective at recognizing variable spellings, but they do not model compositional scope, sarcasm, who holds the opinion, or whether a phrase is quoted. A native-speaker-led error taxonomy would be the most important next analytical step.

7. Deployment

The public system is deployed on GitHub Pages at the live-system link near the start of this report [11]. The build exports the actual learned vocabulary, IDF values, SVM coefficients, and intercepts from the Python joblib artifact into JSON. A JavaScript runtime reproduces normalization, character n-gram counting, sublinear TF-IDF, L2 normalization, linear class scores, and relative-confidence calculation. Text is processed in the visitor's browser. The original Python Streamlit application remains available locally [10]. Both interfaces show influential character patterns rather than claiming semantic understanding.

Layer	Implementation	Reproducibility role
Data	Official TSV files + downloader	Row-count validation; raw data not duplicated in repo
Training	scikit-learn pipelines	Prevents vocabulary leakage across splits
Artifact	Joblib (~1.3 MB); JSON (~3.8 MB)	Same learned parameters in Python and JavaScript
Interface	GitHub Pages; local Streamlit	Text, four examples, prediction, evidence, model card
Testing	pytest + numerical parity + browser	Model equivalence, real interactions, mobile layout

Table 7. Deployment architecture.

Implementation parity was checked on all 5,303 test texts plus nine edge cases: zero label mismatches, zero normalization mismatches, and maximum absolute SVM-margin difference below 1e-10. This is a numerical implementation check without model tuning or new performance claims. The deployment workflow repeats parity before publication. Automated Chrome checks on the public URL passed positive, neutral, negative, known-error, blank-input, and mobile-layout paths with no JavaScript errors. Screenshot evidence is versioned. The student independently recorded and uploaded the demonstration linked at the start of this report.

The interface displays responsible-use notes and labels the margin-derived confidence as uncalibrated. User text is not sent to a prediction service. The roughly 3.8 MB JSON download is larger than the compressed Python artifact, but inference requires no Python server or commercial API. Source pushes to main trigger the GitHub Actions Pages workflow. Local reproduction and fallback instructions are in the README.

8. Limitations and Future Work

- Domain: evaluation covers the released Twitter test split only. Product reviews, radio transcripts, news comments, and customer-service messages may differ substantially.
- Language coverage: Hausa varieties, code-switching practices, orthographic conventions, and emerging slang are not uniformly represented. No language-identification gate is applied.
- Labels: a single positive/neutral/negative label cannot express mixed affect, sarcasm, stance toward multiple targets, or annotator disagreement.
- Method: linear character features do not represent long-range context or compositional negation. Relative SVM confidence is not probability calibration.
- Evaluation: the project did not conduct cross-domain, temporal, dialectal, fairness, robustness, or native-speaker acceptability studies.
- Validation independence: 215 normalized texts overlap training and validation. A supplementary overlap-free comparison confirms the character model's ranking, but does not erase the original design limitation.
- Privacy and governance: public tweets can still contain sensitive content. Raw split files are excluded, but test prediction/error CSVs and quoted examples contain released test text and retain dataset attribution obligations.

Prioritized future work

- Run a native-speaker review of a stratified error sample, separating annotation ambiguity from model failure.
- Fine-tune AfriBERTa or AfroXLM-R under the identical split protocol and report compute, emissions proxy, artifact size, latency, and F1 together.
- Calibrate confidence on a reserved calibration set and add abstention for low-confidence or out-of-domain input.
- Collect consent-aware, naturally occurring evaluation data from at least one non-Twitter domain and a later time period.
- Test robustness to spelling variants, diacritics, code-switching, negation, emoji changes, and adversarial punctuation without changing labels.

9. Conclusion

HausaLens demonstrates a complete, reproducible African-language NLP workflow rather than a web wrapper around an existing chatbot. Using 22,152 human-annotated NaijaSenti Hausa tweets, the project established a majority baseline, compared five learned configurations, selected exclusively on validation macro-F1, evaluated once on the held-out test set, and deployed the fitted artifact. Character 3-5 gram TF-IDF with a class-balanced linear SVM was the strongest local configuration, achieving 0.7649 test macro-F1 and a small 1.3 MB footprint.

The central empirical lesson is that subword form matters: character features outperform word features on this informal dataset, while adding word features provides no gain. The central analytical lesson is that aggregate accuracy conceals a substantial neutral-class weakness and elevated errors around negation and questions. The central engineering lesson is that a modest model can still support transparent, CPU-friendly deployment. The system is useful as an educational baseline and research prototype, but stronger contextual models, probability calibration, cross-domain data, and native-speaker-led evaluation are required before broader use.

10. References

[1] Muhammad, S. H., Adelani, D. I., Ruder, S., et al. (2022). NaijaSenti: A Nigerian Twitter Sentiment Corpus for Multilingual Sentiment Analysis. Proceedings of LREC 2022, 590-602. https://aclanthology.org/2022.lrec-1.63/

[2] Muhammad, S. H., Abdulmumin, I., Yimam, S. M., et al. (2023). SemEval-2023 Task 12: Sentiment Analysis for African Languages (AfriSenti-SemEval). Proceedings of SemEval-2023, 2319-2337. https://doi.org/10.18653/v1/2023.semeval-1.315

[3] Ogueji, K., Zhu, Y., & Lin, J. (2021). Small Data? No Problem! Exploring the Viability of Pretrained Multilingual Language Models for Low-resourced Languages. MRL 2021, 116-126. https://doi.org/10.18653/v1/2021.mrl-1.11

[4] Keinan, R., & Hacohen-Kerner, Y. (2023). JCT at SemEval-2023 Tasks 12 A and 12B: Sentiment Analysis for Tweets Written in Low-resource African Languages. SemEval-2023, 365-378. https://doi.org/10.18653/v1/2023.semeval-1.49

[5] Cortes, C., & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20, 273-297. https://doi.org/10.1007/BF00994018

[6] HausaNLP. (2022). NaijaSenti dataset repository, v1.0. https://github.com/hausanlp/NaijaSenti (accessed 25 September 2026).

[7] scikit-learn developers. (2026). TfidfVectorizer documentation. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html (accessed 25 September 2026).

[8] Salahudeen, S. A., Lawan, F. I., Wali, A. M., et al. (2023). HausaNLP at SemEval-2023 Task 12: Leveraging African Low Resource TweetData for Sentiment Analysis. SemEval-2023, 58-66. https://aclanthology.org/2023.semeval-1.6/

[9] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830. https://jmlr.org/papers/v12/pedregosa11a.html

[10] Streamlit. (2026). Streamlit documentation. https://docs.streamlit.io/ (accessed 25 September 2026).

[11] GitHub. (2026). Using custom workflows with GitHub Pages. https://docs.github.com/en/pages/getting-started-with-github-pages/using-custom-workflows-with-github-pages (accessed 16 October 2026).

[12] ReportLab. (2026). ReportLab documentation. https://docs.reportlab.com/ (accessed 16 October 2026).

Appendix A — Reproducibility and Individual Contribution

A.1 Exact workflow

Step	Command	Output
1	python scripts/download_data.py	Validated official train/dev/test TSV files
2	python scripts/train.py	Six validation runs, final test metrics, model, predictions, figures
3	python scripts/analyze_errors.py	Error slices and length analysis
3a	python scripts/audit_data.py	Missing rows, duplicates, normalized split overlap
3b	python scripts/validate_selection.py	Overlap-free validation sensitivity comparison
4	pytest -q	Automated normalization and inference tests
5	streamlit run app.py	Interactive local web interface
6	python scripts/build_report.py	This report from measured outputs
7	python scripts/export_web_model.py	Learned parameters for public browser inference
8	python scripts/verify_web_model.py	Python/JavaScript numerical-equivalence evidence

Table A1. Reproduction sequence.

A.2 Runtime and artifacts

The reported experiment ran with Python 3.13.0 and scikit-learn 1.8.0 on macOS-26.5.2-arm64-arm-64bit-Mach-O. Deterministic split files were supplied by the dataset publisher; no random resampling was used. The trained model, raw prediction file, error-only file, confusion matrix, experiment table, metrics JSON, and figures are versioned project outputs. The raw tweet TSV files are excluded and can be reacquired with attribution.

A.3 What was implemented in this project

- Implemented: data validation and preprocessing pipeline; six model configurations; metric computation; selection protocol; held-out evaluation; error slicing; visualizations; local explanation utility; Streamlit interface; tests; documentation; PDF builder; demo assets.
- Reused with attribution: NaijaSenti annotations and split files; scikit-learn estimators and vectorizers; Streamlit framework; ReportLab and plotting libraries; published benchmark findings.
- Not used: synthetic training data, commercial sentiment APIs, hosted chatbot inference, or manual alteration of test predictions.
- AI assistance: AI tools assisted coding, documentation, deployment, and verification. The student recorded the personal demonstration and remains responsible for understanding and checking the submitted work and answering the individual technical defense.
- Submission resources: source code, the public application, and the student's recorded demonstration are linked at the start of this report. Demo video: https://youtu.be/g8qN6C-GF6k. The report generator preserves this video link by default; --video-url can override it for a replacement recording.

Verification evidence: outputs/data_audit.json, outputs/validation_sensitivity.csv, outputs/web_parity.json, and outputs/deployment_verification.json. The complete submission checklist and personal recording guide are linked from the README.